```
import pandas as pd
```

```
reviews = pd.read_csv('olist_order_reviews_dataset.csv')
orders = pd.read_csv('olist_orders_dataset.csv')
customer_locations = pd.read_csv('olist_customers_dataset.csv')
```

```
# STORY 1
```

```
reviews["review_score"] = reviews.groupby("order_id")["review_score"].transform("mean")
reviews = reviews.drop_duplicates(subset="order_id")
# some order_ids had more than one review
```

```
reviews.head()
```

	review_id	order_id	review_score	review_comment_title	review_comment_messag
0	7bc2406110b926393aa56f80a40eba40	73fc7af87114b39712e6da79b0a377eb	4.0	NaN	Nal
1	80e641a11e56f04c1ad469d5645fdfde	a548910a1c6147796b98fdf73dbeba33	5.0	NaN	Nal
2	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	5.0	NaN	Nal
3	e64fb393e7b32834bb789ff8bb30750e	658677c97b385a9be170737859d3511b	5.0	NaN	Recebi bem antes d prazo estipulado
4	f7c4243c7fe1938f181bec41a392bdeb	8e6bfb81e283fa7e4f11123a3fb894f1	5.0	NaN	Parabéns lojas lanniste adorei comprar pela I.

Next steps: Generate code with reviews New interactive sheet

```
reviews["order_id"].value_counts().head()
```

order_id	count
90531360ecb1eec2a1fbb265a0db0508	1
5f4a120717a4d152577d7d2ec66c45be	1
10a3f7e9bf88b7dc86a767fd278e5bd2	1
d398e9c82363c12527f71801bf0e6100	1
287445be3e2a5f0d3f5036b2a778d1c5	1

dtype: int64

```
orders.head()
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29

Next steps: Generate code with orders New interactive sheet

```
customer_locations.head()
```

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409	franca	SP
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	9790	sao bernardo do campo	SP
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	1151	sao paulo	SP
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	8775	mogi das cruzes	SP

Next steps: Generate code with customer_locations New interactive sheet

```
orders_reviews = pd.merge(
    orders,
    reviews,
    on="order_id",
    how="left"
)
orders_reviews.head()
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29

Next steps: Generate code with orders_reviews New interactive sheet

```
print(orders.shape[0])
print(orders_reviews.shape[0])
```

```
99441
99441
```

```
master = pd.merge(
    orders_reviews,
    customer_locations,
    on="customer_id",
    how="left"
)
master.head()
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29

Next steps: Generate code with master New interactive sheet

```
print("Orders:", orders.shape[0])
print("Master:", master.shape[0])
```

```
Orders: 99441
Master: 99441
```

```
# so i have basically checked dupplicates by checking to see that my dataset has the same number of rows
```

```
#  now I'm checking for missing values
(master.isna().sum() / len(master)) * 100
```

	0
order_id	0.000000
customer_id	0.000000
order_status	0.000000
order_purchase_timestamp	0.000000
order_approved_at	0.160899
order_delivered_carrier_date	1.793023
order_delivered_customer_date	2.981668
order_estimated_delivery_date	0.000000
review_id	0.772317
review_score	0.772317
review_comment_title	88.384067
review_comment_message	58.985730
review_creation_date	0.772317
review_answer_timestamp	0.772317
customer_unique_id	0.000000
customer_zip_code_prefix	0.000000
customer_city	0.000000
customer_state	0.000000

dtype: float64

```
master = master.drop(columns=["review_comment_title", "review_comment_message"])
# dropped these columns because they had over 50% of their values missing
# left rows of columns with > 3% missing values
```

```
(master.isna().sum() / len(master)) * 100
```

	0
order_id	0.000000
customer_id	0.000000
order_status	0.000000
order_purchase_timestamp	0.000000
order_approved_at	0.160899
order_delivered_carrier_date	1.793023
order_delivered_customer_date	2.981668
order_estimated_delivery_date	0.000000
review_id	0.772317
review_score	0.772317
review_creation_date	0.772317
review_answer_timestamp	0.772317
customer_unique_id	0.000000
customer_zip_code_prefix	0.000000
customer_city	0.000000
customer_state	0.000000

dtype: float64

```
master.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 16 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   order_id                       99441 non-null  object
 1   customer_id                    99441 non-null  object
 2   order_status                   99441 non-null  object
 3   order_purchase_timestamp       99441 non-null  object
 4   order_approved_at              99281 non-null  object
 5   order_delivered_carrier_date   97658 non-null  object
 6   order_delivered_customer_date  96476 non-null  object
 7   order_estimated_delivery_date  99441 non-null  object
 8   review_id                      98673 non-null  object
 9   review_score                   98673 non-null  float64
 10  review_creation_date           98673 non-null  object
 11  review_answer_timestamp        98673 non-null  object
 12  customer_unique_id             99441 non-null  object
 13  customer_zip_code_prefix       99441 non-null  int64
 14  customer_city                  99441 non-null  object
 15  customer_state                 99441 non-null  object
dtypes: float64(1), int64(1), object(14)
memory usage: 12.1+ MB
```

```
# STORY 2
```

```
# converting from str objects to datetime objects to enable calculation of difference in dates
master['order_delivered_customer_date'] = pd.to_datetime(master['order_delivered_customer_date'])
master['order_estimated_delivery_date'] = pd.to_datetime(master['order_estimated_delivery_date'])
```

```
master.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 16 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   order_id                       99441 non-null  object
 1   customer_id                    99441 non-null  object
 2   order_status                   99441 non-null  object
 3   order_purchase_timestamp       99441 non-null  object
 4   order_approved_at              99281 non-null  object
 5   order_delivered_carrier_date   97658 non-null  object
```

```
 6   order_delivered_customer_date  96476 non-null  datetime64[ns]
 7   order_estimated_delivery_date  99441 non-null  datetime64[ns]
 8   review_id                      98673 non-null  object
 9   review_score                   98673 non-null  float64
 10  review_creation_date           98673 non-null  object
 11  review_answer_timestamp        98673 non-null  object
 12  customer_unique_id             99441 non-null  object
 13  customer_zip_code_prefix       99441 non-null  int64
 14  customer_city                  99441 non-null  object
 15  customer_state                 99441 non-null  object
dtypes: datetime64[ns](2), float64(1), int64(1), object(12)
memory usage: 12.1+ MB
```

```
def calculate_delay(df):


  df["days_diff"] = (df["order_delivered_customer_date"] - df["order_estimated_delivery_date"]).dt.days

  df["delivery_status"] = pd.cut(
      df["days_diff"],
      bins=[-1000, 0, 5, 1000],   # edges
      labels=["On Time", "Late", "Super Late"]
  )

  return df
```

```
calculate_delay(master)
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approve
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:0
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:2
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:5
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:4
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:2
...	...	...	...	...	
99436	9c5dedf39a927c1b2549525ed64a053c	39bd1228ee8140590ac3aca26f2dfe00	delivered	2017-03-09 09:54:05	2017-03-09 09:5
99437	63943bddc261676b46f01ca7ac2f7bd8	1fca14ff2861355f6e5f14306ff977a7	delivered	2018-02-06 12:58:58	2018-02-06 13:1
99438	83c1379a015df1e13d02aae0204711ab	1aa71eb042121263aafbe80c1b562c9c	delivered	2017-08-27 14:46:43	2017-08-27 15:0
99439	11c177c8e97725db2631073c19f07b62	b331b74b18dc79bcdf6532d51e1637c1	delivered	2018-01-08 21:28:27	2018-01-08 21:3
99440	66dea50a8b16d9b4dee7af250b4be1a5	edb027a75a1449115f6b43211ae02a24	delivered	2018-03-08 20:57:30	2018-03-09 11:2

99441 rows × 18 columns

Next steps: Generate code with `master` New interactive sheet

```
master["delivery_status"].value_counts().head()
```

delivery_status	count
On Time	89941
Super Late	3765
Late	2770

dtype: int64

```
master["order_status"].value_counts().head(10)
```

order_status	count
delivered	96478
shipped	1107
canceled	625
unavailable	609
invoiced	314
processing	301
created	5
approved	2

dtype: int64

```
# excluded observations/orders with order_statuses of 'cancelled' or 'unavailable'

master = master[~master["order_status"].isin(["canceled", "unavailable"])]
```

```
master["order_status"].shape
```

```
(98207,)
```

```
# STORY 3
```

```
# finding the customer state with the most counts and assuming that to be the distribution center

master["customer_state"].value_counts()
```

customer_state	count
SP	41127
RJ	12698
MG	11496
RS	5417
PR	4983
SC	3600
BA	3344
DF	2121
ES	2018
GO	1998
PE	1643
CE	1323
PA	969
MT	902
MA	736
MS	708
PB	531
PI	490
RN	482
AL	411
SE	345
TO	278
RO	246
AM	147
AC	81
AP	68
RR	45

dtype: int64

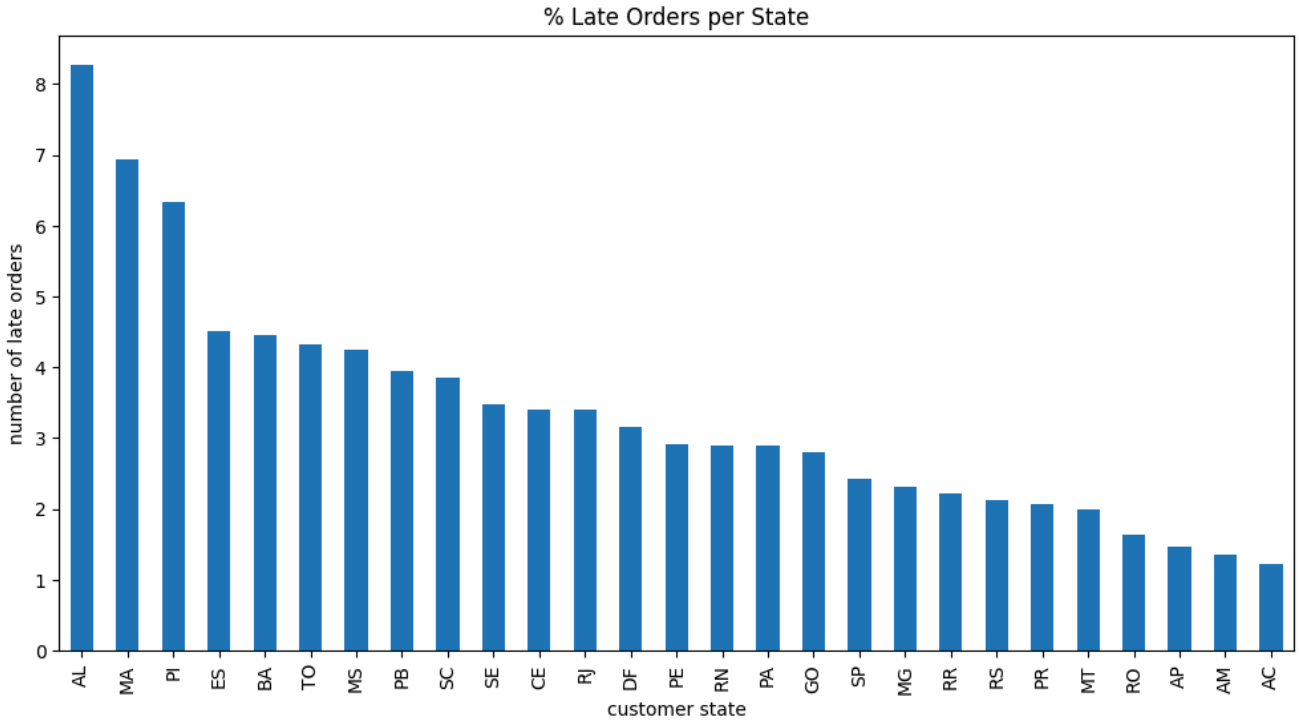
```
# percentage of late orders for each state
late_counts = master[master["delivery_status"] == "Late"].groupby("customer_state")["order_id"].count()

total_counts = master.groupby("customer_state")["order_id"].count()

late_percentage = ((late_counts / total_counts) * 100).round(2).sort_values(ascending=False)

late_percentage
```

customer_state	order_id
AL	8.27
MA	6.93
PI	6.33
ES	4.51
BA	4.46
TO	4.32
MS	4.24
PB	3.95
SC	3.86
SE	3.48
CE	3.40
RJ	3.40
DF	3.16
PE	2.92
RN	2.90
PA	2.89
GO	2.80
SP	2.43
MG	2.31
RR	2.22
RS	2.12
PR	2.07
MT	2.00
RO	1.63
AP	1.47
AM	1.36
AC	1.23

dtype: float64

```
late_percentage.plot(kind="bar",
                     figsize=(12,6),
                     xlabel="customer state",
                     ylabel="number of late orders",
                     title="% Late Orders per State")
```

```
<Axes: title={'center': '% Late Orders per State'}, xlabel='customer state', ylabel='number of late orders'>
```




```
from geopy.geocoders import Nominatim
from geopy.distance import geodesic
```

```
# calculating the approximate distance from the distribution center state to all other states

geolocator = Nominatim(user_agent="state_centroids", timeout=5)

# São Paulo distribution center
sp_coords = (-23.54, -46.64)


states = ["RJ","MG","RS","PR","SC","BA","DF","ES","GO","PE","CE","PA","MT","MA","MS","PB","PI","RN","AL","SE","TO","RO","AM","A

coords = {}
for state in states:
    location = geolocator.geocode(state + ", Brazil")
    if location:
        coords[state] = (location.latitude, location.longitude)



# Compute distances
distances = {state: geodesic(sp_coords, coords[state]).km for state in coords}
```

```
distances
```

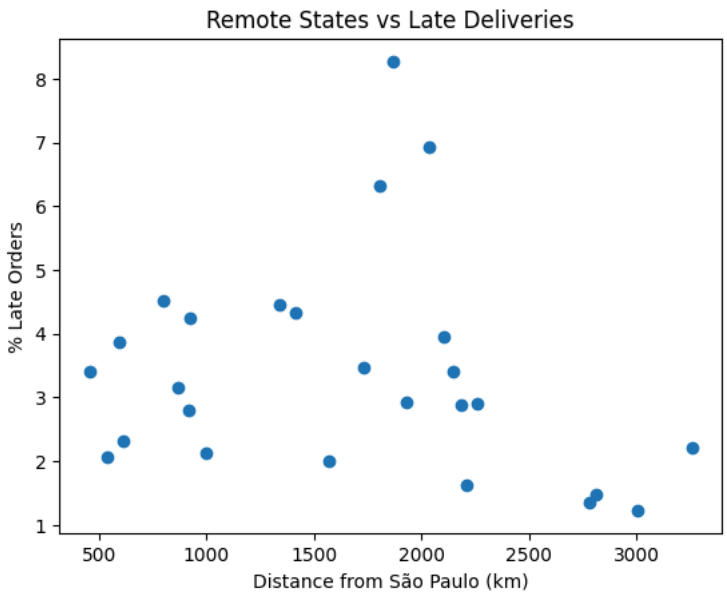
```
{'RJ': 455.06050382222156,
 'MG': 612.0250103139001,
 'RS': 994.9339646111607,
 'PR': 536.7522359514069,
 'SC': 596.0258223825599,
 'BA': 1341.5745756036285,
 'DF': 868.0519422656009,
 'ES': 801.1534509740511,
 'GO': 918.4405298910925,
 'PE': 1932.3605425784672,
 'CE': 2147.8986824874864,
 'PA': 2185.052090354438,
 'MT': 1569.7637174852784,
 'MA': 2032.8024385706644,
 'MS': 922.2439079660451,
 'PB': 2103.503548748251,
 'PI': 1807.74775440597,
 'RN': 2256.829872517369,
```

```
 'AL': 1867.689663677411,
 'SE': 1730.2621588313389,
 'TO': 1412.5364333793243,
 'RO': 2211.39890014486,
 'AM': 2781.1179831814375,
 'AC': 3006.359677964853,
 'AP': 2812.76048394794,
 'RR': 3257.8622453977173}
```

```
distance_df = pd.DataFrame(list(distances.items()), columns=["customer_state", "distance_km"])
late_df = late_percentage.reset_index()

state_analysis = pd.merge(late_df, distance_df, on="customer_state", how="left")
print(state_analysis.sort_values("distance_km", ascending=False))
```

```
   customer_state  order_id  distance_km
19             RR      2.22  3257.862245
26             AC      1.23  3006.359678
24             AP      1.47  2812.760484
25             AM      1.36  2781.117983
14             RN      2.90  2256.829873
23             RO      1.63  2211.398900
15             PA      2.89  2185.052090
10             CE      3.40  2147.898682
7              PB      3.95  2103.503549
1              MA      6.93  2032.802439
13             PE      2.92  1932.360543
0              AL      8.27  1867.689664
2              PI      6.33  1807.747754
9              SE      3.48  1730.262159
22             MT      2.00  1569.763717
5              TO      4.32  1412.536433
4              BA      4.46  1341.574576
20             RS      2.12   994.933965
6              MS      4.24   922.243908
16             GO      2.80   918.440530
12             DF      3.16   868.051942
3              ES      4.51   801.153451
18             MG      2.31   612.025010
8              SC      3.86   596.025822
21             PR      2.07   536.752236
11             RJ      3.40   455.060504
17             SP      2.43          NaN
```

```
import matplotlib.pyplot as plt

plt.scatter(state_analysis["distance_km"], state_analysis["order_id"])
plt.xlabel("Distance from São Paulo (km)")
plt.ylabel("% Late Orders")
plt.title("Remote States vs Late Deliveries")
plt.show()
```



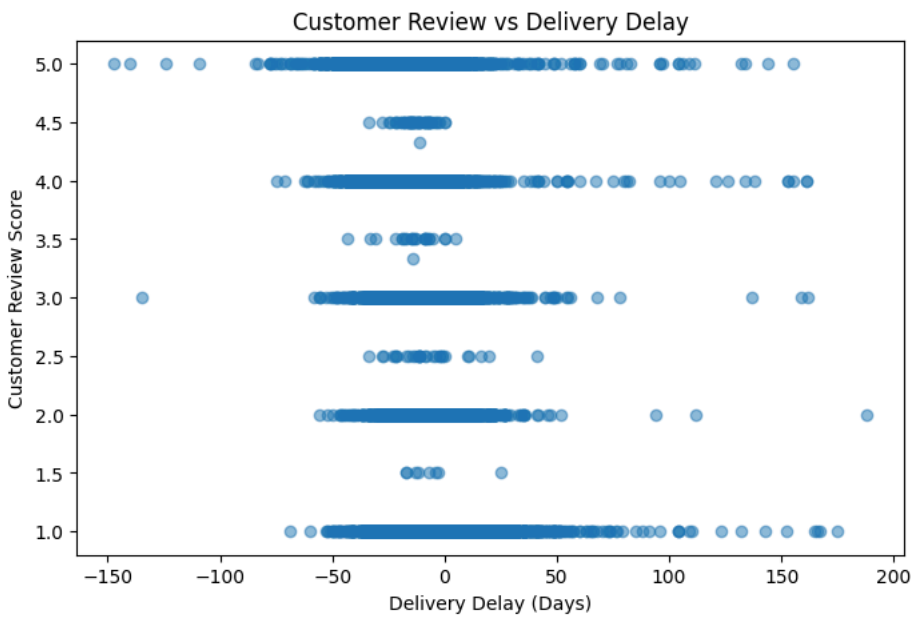
```
# remote states aren't visibly disapproportionately affected
```

```
# STORY 4
```

```
# excluding missing values
sentiment_vs_days = master.dropna(subset=['review_score', 'days_diff'])

sentiment_vs_days[['days_diff', 'review_score', 'delivery_status']].head()
```

	days_diff	review_score	delivery_status
0	-8.0	4.0	On Time
1	-6.0	4.0	On Time
2	-18.0	5.0	On Time
3	-13.0	5.0	On Time
4	-10.0	5.0	On Time

```
# visualising "Delivery Delay (Days)" vs "Average Review Score (1-5)"
plt.figure(figsize=(8,5))
plt.scatter(sentiment_vs_days['days_diff'], sentiment_vs_days['review_score'], alpha=0.5)
plt.xlabel('Delivery Delay (Days)')
plt.ylabel('Customer Review Score')
plt.title('Customer Review vs Delivery Delay')
plt.show()

# so the more negative the delivery delay is (delivered before estimated date or before time), the higher the review
# a slant in the distribution can be noted - as the delay days increase, the smaller the review score is
```




```
avg_review_by_status = sentiment_vs_days.groupby('delivery_status')['review_score'].mean()
avg_review_by_status
```

```
/tmp/ipython-input-3544731271.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a
  avg_review_by_status = sentiment_vs_days.groupby('delivery_status')['review_score'].mean()
```

delivery_status	review_score
On Time	4.290589
Late	2.988795
Super Late	1.738453

dtype: float64

```
# STORY 5 (BONUS)
```

```
customer_products = pd.read_csv('olist_products_dataset.csv')
customer_products.head()
```

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria	40.0	287.0	1.0
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	44.0	276.0	1.0
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.0	250.0	1.0
3	cef67bcfe19066a932b7673e239eb23d	bebes	27.0	261.0	1.0
4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas	37.0	402.0	4.0

Next steps: Generate code with `customer_products` New interactive sheet

```
translation_array = pd.read_csv("product_category_name_translation.csv")
translation_array.head()
```

	product_category_name	product_category_name_english
0	beleza_saude	health_beauty
1	informatica_acessorios	computers_accessories
2	automotivo	auto
3	cama_mesa_banho	bed_bath_table
4	moveis_decoracao	furniture_decor

Next steps: Generate code with `translation_array` New interactive sheet

```
order_items = pd.read_csv("olist_order_items_dataset.csv")
order_items.head()
```

	order_id	order_item_id	product_id	seller_id	shipping
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-0
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-0
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-0
3	00024acbcdf0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-0
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-0

```
# Merging order_items with products under product_id to get category name
order_products = order_items.merge(customer_products, on="product_id", how="left")
order_products.head()
```

	order_id	order_item_id	product_id	seller_id	shipping
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-0
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-0
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-0
3	00024acbcdf0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-0
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-0

```
order_products = order_products.merge(
    translation_array,
    on="product_category_name",
    how="left"
)
order_products.head()
```

	order_id	order_item_id	product_id	seller_id	shipping
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-0
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-0
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-0
3	00024acbcdf0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-0
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-0

```
subset = order_products[["order_id", "product_category_name", "product_category_name_english"]]
subset
```

	order_id	product_category_name	product_category_name_english
0	00010242fe8c5a6d1ba2dd792cb16214	cool_stuff	cool_stuff
1	00018f77f2f0320c557190d7a144bdd3	pet_shop	pet_shop
2	000229ec398224ef6ca0657da4fc703e	moveis_decoracao	furniture_decor
3	00024acbcdf0a6daa1e931b038114c75	perfumaria	perfumery
4	00042b26cf59d7ce69dfabb4e55b4fd9	ferramentas_jardim	garden_tools
...	...	...	...
112645	fffc94f6ce00a00581880bf54a75a037	utilidades_domesticas	housewares
112646	fffcd46ef2263f404302a634eb57f7eb	informatica_acessorios	computers_accessories
112647	fffce4705a9662cd70adb13d4a31832d	esporte_lazer	sports_leisure
112648	fffe18544ffabc95dfada21779c9644f	informatica_acessorios	computers_accessories
112649	fffe41c64501cc87c801fd61db3f6244	cama_mesa_banho	bed_bath_table

112650 rows × 3 columns

```
# STORY 6 - CANDIDATE CHALLENGE
```

```
# Merge order_products with master on order_id = complete dataframe
merged_df = master.merge(
    order_products,
    on="order_id",
    how="left"
)
```